MASTER
GIT & GITHUB

Session Guide and Concepts Primer
From basics to advanced workflows

Session details	
Date	Monday, 12 October 2026
Time	8:00 AM - 12:00 PM (4 hours, with two 10-minute breaks)
Venue	EmpowerWithIT
Entry	Free
Facilitator	Abdibaasit, Software Engineer and Mentor
Audience	Junior and intermediate software engineers and the wider developer fraternity
Format	Short teaching bursts followed by hands-on exercises; roughly 40 percent talk and 60 percent doing

How to use this guide

- **Part 1** is the one-page plan: goals, agenda, materials, and a timing tracker you can fill in on the day.
- **Part 2** is the concepts primer: the history, the essence of Git and GitHub, and every core idea explained in plain language. Read it tonight; dip into it when a question surprises you.
- **Part 3** is the run-sheet: block by block, with talking points, live-demo scripts, activity instructions, check questions, and common pitfalls.
- **Part 4** is the appendix: a script to build the practice repo, an end-of-session quiz with answers, and next-step resources.
- Pair this with the **Command Reference** PDF, which explains every command in detail and can be handed to participants.

Contents

Part	What is inside
Part 1	Session at a glance (goals, agenda, materials, timing tracker)
Part 2	Concepts primer (why version control, history, Git vs GitHub, how Git thinks, core ideas, GitHub features, security, AI)
Part 3	Facilitator run-sheet (Blocks 0 to 5 and wrap-up)
Part 4	Appendix (practice repo script, quiz, checklists, resources)

PART 1

Session at a glance

Learning outcomes

By the end of the session, every participant should be able to:

- Explain what version control is, why Git exists, and how Git and GitHub differ.
- Create a repository, make clean commits, inspect history, and undo common mistakes.
- Create and switch branches, merge, rebase, and resolve a merge conflict confidently.
- Clone, fork, push, open a pull request, and take part in a code review on GitHub.
- Choose a sensible team workflow (Git Flow or trunk-based) and write good commit messages.
- Use stash, cherry-pick, and reflog to rescue and reorganise work, and read a simple GitHub Actions workflow.
- Use AI assistants responsibly to speed up Git and GitHub work without trusting them blindly.

Agenda at a glance

Time	Block	Format	Outcome
8:00 - 8:15	**Block 0** Welcome and setup check	Hook story, poll, config check	Everyone's machine works
8:15 - 9:05	**Block 1** Git fundamentals	Teach 20 / Do 25 / Wrap 5	First repo, commits, undo, branch
9:05 - 9:15	Break		
9:15 - 10:05	**Block 2** Merging, rebasing, conflicts	Teach 15 / Demo 10 / Do 20 / Wrap 5	Resolve a real conflict
10:05 - 10:55	**Block 3** GitHub collaboration	Teach 15 / Do 30 / Wrap 5	A merged pull request each
10:55 - 11:05	Break		
11:05 - 11:30	**Block 4** Team workflows	Teach 10 / Group task 12 / Wrap 3	Pick and defend a workflow
11:30 - 11:55	**Block 5** Advanced tools and AI	Rapid demos with try-it moments	Stash, pick, reflog, Actions, AI
11:55 - 12:00	Wrap-up	Recap, resources, challenge	Clear next steps

Materials checklist

Item	Details
Practice repo	Public GitHub repo **git-masterclass-practice** built with the script in Part 4, including a conflict branch and a CI workflow file
Command Reference PDF	Printed copies if possible, plus a digital copy to share
Projector setup	Terminal font size 18 or larger, light-on-dark theme, VS Code zoomed in
Board or flipchart	For the four-places diagram and commit-graph drawings; marker pens in two colours
Participant setup	Git, VS Code, a GitHub account with two-factor authentication, and a charged laptop
Backup plan	Offline copy of the practice repo on a USB drive and a mobile hotspot, in case the network fails
Helper	A second mentor or confident participant to circulate during exercises

Timing tracker (fill in on the day)

Block	Planned start	Planned end	Actual start	Notes
Block 0 Welcome	8:00	8:15		
Block 1 Fundamentals	8:15	9:05		
Break	9:05	9:15		
Block 2 Merge / rebase	9:15	10:05		
Block 3 GitHub	10:05	10:55		
Break	10:55	11:05		
Block 4 Workflows	11:05	11:30		
Block 5 Advanced + AI	11:30	11:55		
Wrap-up	11:55	12:00		

IF YOU RUN LATE

Protect the hands-on activities and shorten the talking. Cut in this order: (1) Block 4 group task becomes a 3-minute discussion, (2) GitHub Actions becomes a 3-minute demo with the YAML shared afterwards, (3) interactive rebase and extra stash options are skipped. Never cut the reflog rescue or the pull request exercise. They are what people remember.

PART 2

Concepts primer

This part explains the ideas behind the commands. A participant who understands these ideas can work out most commands on their own.

2.1 The problem version control solves

Imagine a project folder that looks like this:

```
report.docx
report_v2.docx
report_final.docx
report_final_REAL.docx
report_final_REAL_use_this_one.docx
```

Now add four teammates, all editing the same files, some emailing zip files, some on a shared drive. Soon nobody knows which version is current, who changed what, why it changed, or how to go back to the version that worked last Tuesday. This is the problem version control solves.

A version control system (VCS) records every change to a set of files over time. With it you can:

- **Go back** to any earlier version, like a time machine for your project.
- **Compare** any two versions and see exactly which lines changed.
- **Know who and why**: every change has an author, a date, and a message explaining it.
- **Work in parallel** without overwriting each other, then combine the work safely.
- **Experiment fearlessly**, because anything can be undone.

Three generations of version control

Generation	Examples	How it works	Weakness
Local	SCCS, RCS	A small database of changes kept on one computer	No collaboration
Centralised	CVS, Subversion (SVN), Perforce	One central server holds the history; developers check files in and out	Needs the network; if the server dies, history may die with it
Distributed	Git, Mercurial	Every developer has a full copy of the entire history	Needs a little more learning at first

2.2 A short history of Git and GitHub

The Linux kernel story. The Linux kernel is one of the largest collaborative projects ever, with thousands of contributors. From 2002 its developers used a proprietary distributed tool called BitKeeper, under a free licence. In April 2005 that arrangement fell apart after a dispute over reverse-engineering, and the free licence was withdrawn. The kernel community suddenly needed a new tool.

Linus Torvalds, the creator of Linux, was not happy with the alternatives available, so he started writing his own, and within days it could manage its own source code. He wanted something that was fast, simple in design, truly distributed, good at handling thousands of parallel branches, and able to cope with a project as big as the kernel. He handed day-to-day maintenance to **Junio Hamano** in the summer of 2005, and Hamano has led the project since.

FUN FACT "Git" is British slang for an unpleasant person. Torvalds joked that he names his projects after himself: first Linux, now Git. The original README described it with several tongue-in-cheek expansions.

Timeline

Year	Milestone
1972	SCCS, one of the first version control tools, appears at Bell Labs
1982	RCS arrives; CVS follows in the late 1980s and 1990
2000	Subversion (SVN) starts as a 'better CVS'; centralised tools dominate the 2000s
2002	The Linux kernel adopts BitKeeper
April 2005	BitKeeper's free licence is withdrawn; Linus Torvalds starts Git; Mercurial begins the same month
Summer 2005	Junio Hamano takes over as maintainer; Git manages the Linux kernel
December 2005	Git 1.0 is released
2008	GitHub launches, built by Tom Preston-Werner, Chris Wanstrath and PJ Hyett (Bitbucket also launches that year)
2011	GitLab begins as an open-source alternative
2018	Microsoft announces its acquisition of GitHub for about 7.5 billion US dollars; GitHub Actions is announced
2019	GitHub Actions becomes generally available; Git 2.23 introduces **git switch** and **git restore**
2020	GitHub and many projects move the default branch name from master to **main**; Git 2.28 adds the init.defaultBranch setting
2021	GitHub stops accepting account passwords for Git operations; personal access tokens or SSH keys are required. GitHub Copilot enters preview
Today	Git is the overwhelming standard for version control; GitHub hosts well over 100 million developers

Why GitHub mattered. Git made collaboration possible technically. GitHub made it *social and easy*: a web page for every repository, the fork-and-pull-request model, issues, profiles, and stars turned code sharing into something anyone could join. That is why open source exploded after 2008, and why a GitHub profile is now part of a developer's CV.

2.3 Git versus GitHub

The most common confusion for beginners. Say this early and repeat it at the end.

SAY IT LIKE THIS Git is the engine. GitHub is the garage. Git is a tool on your computer that tracks changes. GitHub is a website that hosts Git repositories and adds collaboration features around them. It is like email and Gmail: email is the technology, Gmail is one service built on it.

	Git	GitHub
What it is	A version control tool (software)	A cloud platform for hosting Git repositories and collaborating
Created by	Linus Torvalds, 2005	Tom Preston-Werner and co-founders, 2008; owned by Microsoft since 2018
Where it runs	On your computer, from the command line or editor	In the browser, plus the gh command-line tool and apps
Needs internet?	No, except to share changes	Yes
Main job	Record history, branch, merge	Share code, review it, track issues, automate tests, host sites
Can you use one without the other?	Yes, Git works fully offline	No, GitHub exists only to host Git repositories
Alternatives	Mercurial, Subversion (older)	GitLab, Bitbucket, Azure DevOps, Gitea, Codeberg

2.4 The essence: why we use Git and GitHub

If participants remember only one slide, make it this one.

> **SAY IT LIKE THIS** Git gives your code a time machine and parallel universes. GitHub gives it a shared home, a review process, and a robot assistant.

Reason	What it means in practice
A safety net	Every commit is a save point. A mistake costs minutes, not days. Nothing is truly lost.
Fearless experiments	Branches are cheap. Try a risky idea on a branch; delete it if it fails, merge it if it works.
Collaboration	Many people change the same project at once and combine the work safely.
Accountability and memory	Every line has an author, a date, and a message. **git blame** and **git log** answer 'who changed this and why?'
Code review and quality	Pull requests mean a second pair of eyes before code reaches the main branch.
Automation	GitHub Actions runs tests, linters, and deployments on every push automatically.
Backup and portability	Every clone is a full copy. Lose a laptop; clone again.
Open source and career	Contribute to the world's projects; your public repositories and pull requests are your portfolio.
Industry standard	Nearly every software job expects Git. It is a baseline skill, not a bonus.

2.5 How Git thinks

Snapshots, not differences

Many older tools stored a list of changes to each file. Git thinks differently: each commit is a **snapshot** of the whole project at that moment. To stay efficient, files that did not change are simply linked to their previous version. This is why branching and switching in Git are so fast.

Everything is an object with a fingerprint

Git stores four kinds of objects, each identified by a hash (a 40-character fingerprint calculated from its content):

Object	What it stores
Blob	The contents of one file
Tree	A folder: a list of blobs and other trees with their names
Commit	A pointer to a tree (the snapshot), plus the author, date, message, and the **parent commit(s)**
Tag	A permanent, named label on a commit, such as v1.0.0

Because the hash is calculated from the content, any change to a file or to history produces a different hash. This is how Git detects corruption and tampering. Git uses SHA-1 hashes today and is gradually adding SHA-256 support.

History is a chain of commits

Each commit points back to its parent. The result is a graph of commits, often called a DAG (directed acyclic graph), which is a fancy way of saying 'a family tree that only points backwards'.

```
A---B---C---D       <- main
         \
          E---F     <- feature/login
```

Branches and HEAD are just pointers

- A **branch** is a tiny file containing the hash of one commit. Creating a branch copies nothing; it just adds a label. That is why branches are cheap.
- **HEAD** is a pointer to 'where you are now', usually to the current branch. When you commit, the current branch label moves forward.
- A **detached HEAD** means HEAD points straight at a commit instead of a branch. Not dangerous, but commits made there can be lost unless you create a branch.

The hidden .git folder

Running git init creates a **.git** folder. That folder *is* the repository: the objects, branches, configuration, and staging information. Delete it and the project is just ordinary files again. Copy it and you have copied the entire history.

Distributed means every clone is complete

When you clone a repository you receive all of its history, not just the latest files. You can commit, branch, view history, and compare versions with no internet at all. GitHub is simply one agreed place where people share their copies.

2.6 The four places and the commit cycle

```
WORKING DIR --git add--> STAGING --git commit--> LOCAL REPO --git push--> REMOTE (GitHub)
(your files)             (next commit)           (.git history)            (shared copy)

     <----------------------- git pull / git fetch -----------------------------------
```

Place	Think of it as	Key commands
Working directory	Your desk, where you are actively editing	edit files, git status, git diff
Staging area (index)	A box where you pack exactly what goes into the next parcel	git add, git restore --staged
Local repository	Your private, complete history	git commit, git log, git branch
Remote repository	The shared copy everyone agrees on	git push, git pull, git fetch, git clone

SAY IT LIKE THIS Why do we have a staging area? Because a day's work often contains two unrelated changes. Staging lets you commit them separately, so each commit tells one clear story.

2.7 Branching, merging, and rebasing

Why branch

A branch is a safe, separate line of development. The standard habit: **one branch per task**. Main stays stable; your messy work happens on your branch until it is ready.

Merge

A merge combines the history of two branches. If the target branch has not moved, Git just slides the pointer forward (a **fast-forward**). If both branches have new commits, Git creates a **merge commit** with two parents.

```
BEFORE                                AFTER GIT MERGE (merge commit)

          A---B---C   feature                     A---B---C
         /                                       /         \
D---E---F---G---H     main            D---E---F---G---H---M   main
```

Rebase

A rebase takes your commits and **replays** them on top of another branch, as if you had started your work from the latest commit. The result is a straight, tidy history. The commits are rewritten, so they get new hashes (shown as A', B', C').

```
AFTER GIT REBASE main (while on feature)

D---E---F---G---H   main
                \
                 A'--B'--C'   feature
```

	Merge	Rebase
History	Preserves exactly what happened, including the branching	Rewrites commits into a clean straight line
Safe on shared branches?	Yes	**No.** Never rebase commits others already have
Typical use	Bringing a finished feature into main	Updating your own feature branch with the latest main; tidying commits before a PR
Downside	Can make a busy graph	Rewriting history confuses teammates if misused

Merge conflicts

A conflict is not an error; it is Git asking a human to decide. It happens when two branches changed the **same lines** of the same file (or one deleted a file the other edited). Git cannot guess whose version wins, so it marks the file and waits.

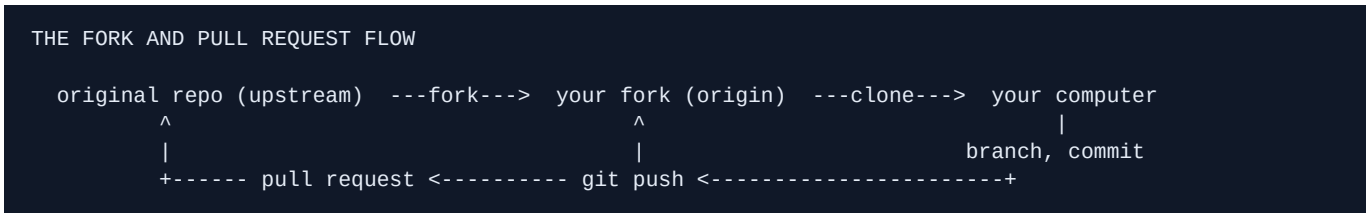
```
<<<<<<< HEAD
const greeting = "Hujambo";        <- what is on the branch you are on
=======
const greeting = "Habari";         <- what is coming in from the other branch
>>>>>>> conflict-demo
```

1. Run git status to see which files are conflicted.
2. Open each file. Decide: keep yours, keep theirs, keep both, or write something new.
3. Delete all three marker lines (the <<<<<<<, =======, and >>>>>>> lines).
4. Run git add on the file to say 'resolved', then git commit (or git rebase --continue) to finish.

> **FACILITATOR TIP** Remove the fear. Say: 'Conflicts are normal. They only mean two people were busy. Small, frequent merges make small, easy conflicts.'

2.8 Remotes, forks, and pull requests

- **Remote**: a Git repository hosted elsewhere. The default name for the one you cloned from is **origin**.
- **Clone**: download a full copy of a remote repository and set it up as origin.
- **Fetch**: download new commits from the remote without changing your files. Safe and reversible.
- **Pull**: fetch plus merge (or rebase). Updates your current branch.
- **Push**: upload your local commits to the remote.
- **Fork**: your own copy of someone else's repository on GitHub. Used when you do not have write permission, which is how open source works.
- **Upstream**: the original repository that your fork came from.
- **Pull request (PR)**: a proposal to merge your branch into another, with a space for discussion, review, and automated checks.



What makes a pull request good

- **Small and focused**: one purpose per PR. Reviewers cannot review 3,000 lines well.
- **A clear title and description**: what changed, why, how to test it, and screenshots for visual changes.
- **Linked to an issue**: for example 'Closes #12' closes the issue automatically on merge.
- **Passing checks**: tests and linters green before asking for review.

What makes a good review

- Review the code, not the person. Be specific, kind, and curious: 'What happens if the list is empty?'
- Separate must-fix problems from optional suggestions (a common convention is to prefix the latter with 'nit:').
- Praise good work. Approve promptly when it is ready; do not let PRs go stale.

2.9 What GitHub adds beyond hosting

Feature	What it does
Repositories	Public or private homes for projects, each with a README that acts as its front page
Issues	Track bugs, tasks, and ideas, with labels, assignees, and milestones
Pull requests	Propose, discuss, review, and merge changes
Projects	Kanban-style boards and tables to plan work
Actions	Automation (CI/CD): run tests, build, and deploy on events such as a push or a PR
Pages	Free static website hosting straight from a repository
Releases	Package and publish versions, attached to tags
Discussions	Forum-style conversations separate from bug tracking
Codespaces	A full development environment in the browser
Dependabot and security	Automatic dependency updates, vulnerability alerts, and secret scanning
Branch protection and rulesets	Require reviews and passing checks before code can reach main
Copilot	AI assistance in the editor and in GitHub itself
Profile and README	Your public developer portfolio: pinned repositories, contribution graph, profile README

CI/CD in one minute

Continuous Integration (CI): every time someone pushes code, an automated system builds it and runs the tests, so problems are caught within minutes. **Continuous Delivery or Deployment (CD)**: code that passes the checks is automatically prepared for release, or released. GitHub Actions does both using YAML files in **.github/workflows/**.

2.10 Signing in: HTTPS tokens and SSH keys

To push to GitHub you must prove who you are. Since 2021, GitHub no longer accepts your account password for Git operations. There are two common ways:

Method	How it works	Good for
HTTPS with a token	You use a Personal Access Token (or the GitHub CLI / credential manager) instead of a password	Quick start; works behind strict networks
SSH key	You create a key pair; the private key stays on your machine and the public key is added to GitHub	Daily use; no repeated prompts

```
# Create an SSH key (press Enter to accept defaults)
ssh-keygen -t ed25519 -C "you@example.com"

# Show the PUBLIC key, then paste it in GitHub: Settings > SSH and GPG keys > New SSH key
cat ~/.ssh/id_ed25519.pub

# Test the connection
ssh -T git@github.com

# Alternative: sign in with the GitHub CLI
gh auth login
```

WATCH OUT Never share the private key (the file without .pub) and never commit it. Turn on two-factor authentication on every GitHub account.

2.11 Good habits that separate beginners from professionals

Commits

- Small, focused, and complete: one idea per commit; the project should still work after each.
- Message format: **type(scope): summary** in the imperative ('add', not 'added'); explain *why* in the body when it is not obvious.
- Never commit generated folders, dependencies (node_modules), or secrets. Use .gitignore.

Branches

- Short-lived and descriptive: feature/login-form, fix/cart-total, docs/readme-setup, chore/update-deps.
- Delete a branch once it is merged. Keep main always working.

Repository hygiene

File	Purpose
README.md	What the project is, how to install it, how to run it. The first thing anyone reads
.gitignore	Patterns of files Git should never track (build output, dependencies, .env, OS files)
LICENSE	Tells others what they may do with your code (common choices: MIT, Apache 2.0, GPL). Without one, code is legally 'all rights reserved'
CONTRIBUTING.md	How others can help: setup, branching rules, commit style
.github/PULL_REQUEST_TEMPLATE.md	A checklist that appears in every new pull request
CODEOWNERS	Automatically requests review from the right people for certain folders

2.12 Security essentials

- **Secrets never go in Git.** That means API keys, passwords, tokens, .env files, and private keys. Git remembers everything, so deleting a file in a later commit does **not** remove it from history.
- If a secret is committed or pushed, **rotate it immediately** (generate a new one and revoke the old one). Only then clean the history (git filter-repo or BFG).
- Keep secrets in environment variables locally and in **GitHub Actions secrets** in the cloud.
- Enable two-factor authentication, secret scanning, and Dependabot alerts.
- Consider signing commits (GPG or SSH) so they show as **Verified** on GitHub.
- A private repository is not a safe place for secrets either: people leave teams, repositories get made public, and tokens leak.

2.13 Using AI with Git and GitHub, responsibly

Good uses	Rules of the road
Draft a commit message from a diff	**Read it.** You own every commit, whoever or whatever wrote it
Write a PR description or summarise a long change	Check that it is accurate and complete before posting
Explain a confusing merge conflict or an error message	Verify the suggested command in a safe place (a throwaway branch) before running it
Draft a GitHub Actions workflow or a .gitignore	Review every step; never let a workflow expose secrets
Suggest a command for a tricky situation (for example recovering a lost commit)	Be wary of destructive commands such as reset --hard, clean -fd, and push --force

GOLDEN RULE Never paste secrets, private keys, customer data, or code you are not allowed to share into an AI tool. AI can speed you up, but it does not replace understanding. If you cannot explain what a command does, do not run it on important work.

2.14 Myths and misunderstandings to correct

Myth	Reality
Git and GitHub are the same thing.	Git is the tool; GitHub is one platform that hosts Git repositories.
A commit stores only the changes.	A commit stores a snapshot of the whole project (efficiently), and Git computes differences when you ask.
Creating a branch copies all my files.	A branch is a tiny pointer to a commit. It costs almost nothing.
Deleting a file in a new commit removes it forever.	It is still in history. That is why leaked secrets must be rotated.
git pull and git fetch are the same.	Fetch only downloads. Pull downloads *and* integrates into your branch.
Force push is always evil.	It is dangerous on shared branches, but --force-with-lease on your own branch after a rebase is routine.
If I run reset --hard, my work is gone.	Committed work is almost always recoverable through git reflog for roughly 90 days.
Git is only for code.	It works for any text: documentation, configuration, writing, even this guide.
Merge conflicts mean I did something wrong.	They are normal and simply need a human decision.

Glossary

Term	Meaning
Repository (repo)	A project folder tracked by Git, including its full history
Commit	A saved snapshot with an author, date, message, and parent
Hash (SHA)	The unique fingerprint of a commit or object, such as 3f2a9c1
Branch	A movable label pointing at a commit
HEAD	Pointer to your current position, usually the current branch
Working tree	The actual files on disk that you edit
Index / staging area	Where changes wait to be included in the next commit
Merge / Rebase	Two ways to combine work from one branch into another
Fast-forward	A merge where Git just moves the pointer forward, with no merge commit
Conflict	Overlapping edits that Git cannot combine automatically
Remote / origin / upstream	A hosted copy / the default remote name / the original repo your fork came from
Tracking branch	A local branch linked to a remote branch, so push and pull know where to go
Stash	A temporary shelf for unfinished changes
Cherry-pick	Copy one commit onto another branch
Reflog	A private log of everywhere HEAD has been; the safety net
Tag / Release	A permanent label on a commit / a published version on GitHub
Issue / Pull request	A tracked task or bug / a proposed change for review
CI/CD	Automated testing and delivery on every change
Workflow / runner	A GitHub Actions automation file / the machine that executes it
Protected branch	A branch with rules, such as required reviews and passing checks
Squash	Combine several commits into one

PART 3

Facilitator run-sheet

Each block follows the same pattern: **objectives, talk track, live demo, activity, check for understanding, pitfalls.** Times are targets. Keep a visible clock, and speak in short bursts of ten minutes or less before getting people typing.

BLOCK 0 Welcome and setup check

8:00 - 8:15
15 minutes

Objectives

- Everyone's machine works.
- People feel welcome and know what the day looks like.
- You know the room's experience level.

Run of show

Min	What to do
3	**Hook story.** 'Friday, 5 pm. Two developers edit the same file. Monday morning, one developer's work is gone and nobody knows why.' Ask: who has lived this?
2	**Poll by show of hands.** Never used Git? Used it alone? Used it in a team? Used pull requests? Note the numbers; it tells you how to pace.
2	**Agenda and ground rules.** Show the five blocks. Ground rules: questions any time, no question is silly, helpers will walk around, type along rather than copy-paste.
8	**Setup check.** Everyone runs the commands below. Helpers fix problems now.

```
git --version
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
```

WATCH OUT Someone will have a missing install or a wrong email. Fix it now. A broken setup at 9:30 costs far more. If a person cannot install Git, pair them with a neighbour for the first block.

BLOCK 1 Git fundamentals

8:15 - 9:05
50 minutes

Objectives

- Explain the four places and the add-commit cycle.
- Create a repo, make good commits, read history, and undo simple mistakes.
- Create a branch and see that switching branches changes the files.

Talk track (about 20 minutes)

- **Why version control** (3 min): the report_final_REAL.docx story from Part 2.1. Introduce Git as the standard answer and mention its history in two sentences (Linus, 2005, the Linux kernel).
- **Git vs GitHub** (2 min): engine and garage. Promise that GitHub comes later in the day.
- **The four places** (5 min): draw working directory, staging, local repo, remote on the board. Walk one change through them with git add and git commit. Leave the drawing up all day.
- **Core commands** (7 min): init, status, add, commit, log --oneline, diff, .gitignore.
- **Branches** (3 min): a branch is a sticky note on a commit, nothing more.

SAY IT LIKE THIS A commit is a save point in a video game. Staging is packing the box before you post it: you choose exactly what goes in. And git status is your map; check it constantly.

Live demo script

```
mkdir demo && cd demo
git init
git status                              # nothing yet
echo "Hello Git" > hello.txt
git status                              # untracked
git add hello.txt
git status                              # staged
git commit -m "feat: add hello file"
git log --oneline

echo "Second line" >> hello.txt
git diff                                # see the change
git add -p                              # stage by chunk
git commit -m "docs: add second line"

# Undo 1: throw away an edit
echo "oops" >> hello.txt
git restore hello.txt

# Undo 2: undo a bad commit but keep the work
echo "bad idea" >> hello.txt && git commit -am "wip: bad idea"
git reset --soft HEAD~1
git restore --staged hello.txt && git restore hello.txt

# Branching
git switch -c feature/greeting
echo "Habari" > greeting.txt && git add . && git commit -m "feat: add greeting"
git switch main                         # greeting.txt vanishes!
git switch feature/greeting             # ...and returns
git log --oneline --graph --all
```

Participant activity (25 minutes)

1. Create a folder called **my-first-repo**, run git init, and add a file with some text.
2. Make **three commits** with meaningful messages. Check each step with git status and git log --oneline.
3. Edit a file and run git diff, then undo the edit with git restore.
4. Make a deliberately bad commit and undo it with git reset --soft HEAD~1.
5. Add a .gitignore containing ***.log**, create a test.log file, and confirm Git ignores it.

6. Create a branch, commit on it, switch back to main, and watch the file disappear and reappear.
7. **Stretch:** use git add -p to commit two unrelated changes as two separate commits.

Check for understanding

- 'Which of the four places is the staging area, and why does it exist?'
- 'I edited a file and want to throw it away. Which command?' (git restore)
- 'Does creating a branch copy my files?' (No: a pointer)

WATCH OUT Common slips: committing before staging (nothing to commit), forgetting quotes around the message (opens an editor; teach :wq or use the VS Code editor setting), and running git init inside another repo. Teach the habit: **status, then act, then status again.**

BLOCK 2 Merging, rebasing, and conflicts

9:15 - 10:05
50 minutes

Objectives

- Distinguish fast-forward and three-way merges.
- Explain what rebase does and the golden rule about shared history.
- Read conflict markers and resolve a conflict without panic.

Talk track (about 15 minutes)

- **Recap** (2 min): ask someone to explain the four places in their own words.
- **Merge** (5 min): draw the fast-forward and the merge-commit pictures from Part 2.7. Show git log --oneline --graph --all before and after.
- **Rebase** (5 min): redraw the same branches after a rebase; point out the new hashes (A', B', C'). State the golden rule: **never rebase commits other people already have.**
- **Conflicts** (3 min): normal, not an error. Show what the markers mean on a slide.

SAY IT LIKE THIS

Branches are parallel tracks. Merge joins the tracks and keeps both stories. Rebase rewrites my track so it looks as if I started from the newest station. A conflict is two people editing the same sentence; Git just needs a human to choose.

Live demo script (about 10 minutes)

```
git clone https://github.com/<you>/git-masterclass-practice.git
cd git-masterclass-practice
git switch main

# 1) A merge that conflicts
git merge conflict-demo                # CONFLICT in app/hello.js
git status                             # both modified
# open app/hello.js in VS Code, use Accept Current / Incoming / Both
git add app/hello.js
git commit                             # finishes the merge
git log --oneline --graph --all

# 2) Rebase
git switch -c feature/rebase-demo
echo "from feature" >> notes.txt && git commit -am "feat: note from feature"
git switch main
echo "from main" >> README.md && git commit -am "docs: note from main"
git switch feature/rebase-demo
git rebase main
git log --oneline --graph --all        # straight line now

# 3) Bail-out buttons
git merge --abort      # while merging
git rebase --abort     # while rebasing
```

Participant activity (20 minutes)

1. **Pair up.** Partner A and Partner B each clone their own copy of the same repo (or use one shared fork).
2. Both create a branch from main and edit **the same line** of app/hello.js differently, then commit.
3. Partner A merges their branch into main. Partner B tries to merge theirs and gets a conflict.
4. Partner B resolves the conflict, completes the merge, and checks the graph with git log --oneline --graph --all.
5. Swap roles and repeat once, this time using **git rebase main** and git rebase --continue.
6. **Stretch:** try git merge --no-ff and compare the graph with a fast-forward.

Check for understanding

- 'Why must we not rebase a branch my teammate has already pulled?'
- 'What do the three marker lines mean, and what must you do to them?'
- 'You are stuck mid-rebase and scared. What do you type?' (git rebase --abort)

FACILITATOR TIP	Make the first conflict happen in front of everyone, on purpose. When people see the red text and then watch it resolved in under a minute, the fear disappears.

BLOCK 3 GitHub collaboration

10:05 - 10:55
50 minutes

Objectives

- Connect a local repo to GitHub and keep both in sync.
- Fork, branch, push, and open a good pull request.
- Give and receive a simple code review.

Talk track (about 15 minutes)

- **GitHub in one sentence** (2 min): 'A home for Git repositories, plus a collaboration layer: issues, reviews, automation.' Recall engine and garage.
- **Remotes** (4 min): origin, clone, push, fetch, pull. Draw the arrows on the four-places diagram. **Fetch looks; pull looks and integrates.**
- **Fork and PR flow** (5 min): draw the fork-clone-branch-push-PR diagram from Part 2.8.
- **Good PRs and reviews** (4 min): small, clear, linked to an issue; review the code, not the person; use 'nit:' for optional comments.

SAY IT LIKE THIS A pull request is not 'merge my code now'. It is 'here is my proposal; please look, comment, and then let us decide together.'

Participant activity (30 minutes): the centrepiece

1. Open the practice repo on GitHub and click **Fork**.
2. Clone **your fork**: git clone <your-fork-url>, then cd into it.
3. Create a branch: git switch -c add-<your-name>.
4. Edit **CONTRIBUTORS.md** and add your name, role, and one fun fact. Commit with: docs: add <your-name> to contributors.
5. Push: git push -u origin add-<your-name>.
6. On GitHub, click **Compare and pull request**. Write a clear title and a two-line description. Create the PR.
7. The facilitator and helpers review live: leave one comment or suggestion on several PRs, approve, and merge.
8. **Expect conflicts:** everyone edited the same file, so later PRs will conflict. Show how to fix it: update the branch from main (merge or rebase), resolve, push again.
9. **Stretch:** review a neighbour's PR and leave a constructive comment; set up the upstream remote and sync your fork.

```
# Keep your fork up to date with the original
git remote add upstream https://github.com/<owner>/git-masterclass-practice.git
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```

Check for understanding

- 'What is the difference between fetch and pull?'
- 'Why do open-source contributors fork instead of pushing directly?'
- 'Name three things a reviewer will thank you for in a PR.'

WATCH OUT Authentication problems are the number one time-waster here. Have helpers ready with 'gh auth login' or a personal access token. Also watch for people pushing straight to main of the original repo: show them how to create a branch and open a PR instead.

BLOCK 4 Team workflows

11:05 - 11:30
25 minutes

Objectives

- Compare Git Flow and trunk-based development.
- Explain why branch protection and Conventional Commits matter in teams.
- Choose a workflow for a given team and justify it.

Talk track (about 10 minutes)

- **Why teams need rules** (2 min): without a shared workflow, five developers produce five habits. A workflow is traffic rules for code.
- **Git Flow** (3 min): main, develop, feature/*, release/*, hotfix/*. Planned releases; good when multiple versions are supported. Weakness: long-lived branches and painful merges.
- **Trunk-based** (3 min): everyone merges into main in small pieces, often daily; feature flags hide unfinished work; relies on strong automated tests. Used by many fast-moving teams.
- **Guard rails** (2 min): branch protection (require PR, reviews, passing checks, no force push to main) and Conventional Commits (feat, fix, docs, refactor, test, chore, ci).

Group activity (12 minutes)

Form 4 or 5 groups. Each gets one scenario card, chooses a workflow, lists two branch-protection rules, and defends the choice in two minutes.

Scenario	Hints to listen for
A 3-person startup shipping to production several times a day	Trunk-based, feature flags, strong CI, short-lived branches
A bank releasing quarterly, with audits and a release manager	Git Flow or release branches, protected branches, mandatory reviews, signed commits
A mobile app supporting three old versions at once	Release branches per version, hotfix branches, cherry-picking fixes
A student open-source project with 20 casual contributors	Fork and PR model, CONTRIBUTING.md, required review by a maintainer, issue templates
A data team versioning notebooks and config files	Simple feature-branch flow, PR reviews, .gitignore for outputs, notebook diffs

Check for understanding

- 'Why does trunk-based development depend on good automated tests?'
- 'Write a Conventional Commit for fixing a crash when the cart is empty.' (fix(cart): handle empty cart on checkout)

SAY IT LIKE THIS

There is no best workflow. The best workflow is the one your team will actually follow, matched to how often you release.

BLOCK 5 Advanced tools and AI

11:30 - 11:55
25 minutes

Objectives

- Use stash, cherry-pick, and reflog to handle real-life interruptions and mistakes.
- Read a simple GitHub Actions workflow and see it run.
- Use AI helpers productively and safely with Git and GitHub.

This block is demo-driven and fast. Each item is 4 to 6 minutes: show it, then let people try it for one minute.

1. Stash: the interruption drawer (4 min)

```
echo "half-finished idea" >> notes.txt
git switch main                 # may be blocked or messy with local edits
git stash push -m "wip: idea"
git switch main                 # clean switch
git stash list
git switch -                    # back to your branch
git stash pop
```

SAY IT LIKE THIS You are cooking when the phone rings. Stash puts the pan in the drawer so you can answer, then you take it out exactly as it was.

2. Cherry-pick: the hotfix move (4 min)

```
git switch main
git switch -c hotfix/typo
echo "fixed typo" >> README.md && git commit -am "fix: correct typo in README"
git log --oneline -1            # copy the hash
git switch release              # any other branch
git cherry-pick <hash>          # copies just that one commit
```

3. Reflog: the black-box recorder (6 min)

The most memorable demo of the day. Do it slowly and dramatically.

```
git log --oneline               # show 4 good commits
git reset --hard HEAD~2         # 'oh no!' - two commits vanish
git log --oneline               # they are gone... or are they?
git reflog                      # find the hash from before the reset
git branch rescue <hash>        # or: git reset --hard <hash>
git log --oneline rescue        # everything is back
```

SAY IT LIKE THIS Git almost never really deletes anything you committed. The reflog is the black-box flight recorder: it remembers everywhere you have been, even when you think you have lost your work.

WATCH OUT Reflog only helps for committed work. Uncommitted changes destroyed by reset --hard or restore are gone. Say this clearly; it is the reason to commit early and often.

4. GitHub Actions: the robot assistant (6 min)

```
mkdir -p .github/workflows
# paste the ci.yml from the Command Reference (Section 16)
git add .github && git commit -m "ci: add test workflow"
git push
# Open the Actions tab on GitHub: watch it run, show the green check
# Break a test, push again, show the red cross, fix, push, green again
```

Explain the four words: **workflow** (the file), **on** (the trigger), **job** (a group of steps on one machine), **step** (one command or action). Then show how branch protection can block merging until the check is green.

5. Guided use of AI (5 min)

- Paste a small diff into an AI assistant and ask for a Conventional Commit message. Read it aloud and edit it together.
- Ask it to explain a conflict block or an error message such as 'rejected non-fast-forward'.
- Ask it to draft a PR description, then check what it got wrong or left out.
- State the rules from Part 2.13: read everything, test commands on a throwaway branch, avoid destructive commands you do not understand, never paste secrets.

Check for understanding

- 'You reset --hard and lost two commits. What command shows where they went?'
- 'When would you use cherry-pick instead of merge?'

BLOCK W Wrap-up

11:55 - 12:00
5 minutes

Run of show

1. **One-sentence recap per block:** fundamentals (edit, stage, commit); merge and rebase (combine work, resolve conflicts); GitHub (fork, PR, review); workflows (agree on rules); advanced (stash, cherry-pick, reflog, Actions, AI).
2. **Return to the hook:** 'Remember Friday 5 pm and the lost work? Which tools from today would have saved it?'
3. **Share resources:** the Command Reference PDF, the practice repo link, and the resource list in Part 4.
4. **Next steps challenge:** make one pull request to a real open-source project within two weeks; pin a repository on your GitHub profile; practise interactive rebase.
5. **Feedback:** ask for one thing that worked and one thing to improve (a short form or sticky notes).

SAY IT LIKE THIS	Git is not about memorising commands. It is about never being afraid of changing your code again. Commit early, commit often, and when in doubt, check git status and git reflog.

PART 4

Appendix

A. Build the practice repo (run tonight)

This script creates the practice repository with a conflict branch, a notes file, a contributors file, and a CI workflow. It assumes Git and the GitHub CLI are installed and you are signed in (gh auth login). Run it in an empty folder.

```
mkdir git-masterclass-practice && cd git-masterclass-practice
git init -b main

cat > README.md <<'EOF'
# Git Masterclass Practice Repo
Practice repository for the EmpowerWithIT Master Git & GitHub class.

## Tasks
1. Add your name to CONTRIBUTORS.md in a pull request.
2. Resolve the conflict between main and the conflict-demo branch.
EOF

printf '# Contributors\n\nAdd your name, role and a fun fact below.\n\n' > CONTRIBUTORS.md
printf 'Class notes\n' > notes.txt
printf '.DS_Store\nnode_modules/\n*.log\n.env\n' > .gitignore

mkdir app
printf 'const greeting = "Hello";\nconsole.log(greeting);\n' > app/hello.js

mkdir -p .github/workflows
cat > .github/workflows/ci.yml <<'EOF'
name: CI
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: node app/hello.js
EOF

git add .
git commit -m "chore: initial practice repo"

# Build the deliberate conflict
git switch -c conflict-demo
printf 'const greeting = "Habari";\nconsole.log(greeting);\n' > app/hello.js
git commit -am "feat: greet in Swahili (Habari)"

git switch main
printf 'const greeting = "Hujambo";\nconsole.log(greeting);\n' > app/hello.js
git commit -am "feat: greet in Swahili (Hujambo)"

# Publish to GitHub (both branches)
gh repo create git-masterclass-practice --public --source=. --remote=origin --push
git push origin conflict-demo
```

FACILITATOR TIP Open the repo in a private browser window afterwards to confirm it is public and that both branches are visible. Test the conflict yourself once: git switch main and git merge conflict-demo should conflict in app/hello.js.

B. Checklists

Tonight

- Create and test the practice repo; confirm the conflict happens.
- Print the Command Reference or prepare to share it; test your projector and terminal font size.
- Send participants the setup checklist: Git, VS Code, a GitHub account with 2FA, a charged laptop.
- Prepare scenario cards for Block 4 and the feedback question.
- Charge your laptop; pack a charger, adapter, markers, and a mobile hotspot.

On the morning

- Arrive early; check network, projector, and board; write the agenda on the board.
- Open a terminal, VS Code, and the practice repo in tabs; zoom everything in.
- Brief your helper on authentication fixes and where the stretch tasks are.

After the session

- Share the PDFs and practice repo link; ask participants to star the repo or follow you.
- Collect feedback; note which block ran over, and adjust the plan for next time.

C. End-of-session quiz (with answers)

Use it as a spoken round, a show-of-hands quiz, or print it as a short take-home.

#	Question	Answer
1	What is the difference between Git and GitHub?	Git is the version control tool on your computer; GitHub is a platform that hosts Git repositories and adds collaboration features.
2	What does the staging area let you do?	Choose exactly which changes go into the next commit, so each commit is focused.
3	Which command undoes the last commit but keeps the changes staged?	git reset --soft HEAD~1
4	Fetch versus pull?	Fetch downloads new commits without changing your files; pull fetches and then merges or rebases into your branch.
5	When must you not rebase?	On commits other people already have, such as shared or public branches.
6	What is a pull request?	A proposal to merge one branch into another, with discussion, review, and automated checks.
7	You ran reset --hard and lost commits. How do you find them?	git reflog to find the earlier hash, then git branch rescue or git reset --hard .
8	What does git stash do?	Shelves uncommitted changes temporarily so you have a clean working directory; git stash pop brings them back.
9	What belongs in .gitignore?	Files Git should never track: dependencies, build output, logs, .env files, and OS junk.
10	Why is push --force-with-lease safer than push --force?	It refuses to overwrite the remote if someone else has pushed since you last fetched.
11	What do you do first if you accidentally push a password?	Rotate (replace) the password immediately; then clean the history.

#	Question	Answer
12	Name two differences between Git Flow and trunk-based development.	Git Flow uses long-lived branches and planned releases; trunk-based uses short-lived branches, daily merges, and relies on strong tests and feature flags.

D. Resources for next steps

Resource	Why it helps
Pro Git (free book) at git-scm.com/book	The definitive, free reference, written by Scott Chacon and Ben Straub
Learn Git Branching at learngitbranching.js.org	A visual, interactive game that makes branching, merging, and rebasing click
GitHub Skills at skills.github.com	Free, hands-on courses that run inside GitHub itself
GitHub Docs at docs.github.com	Authoritative guides for every GitHub feature
Conventional Commits at conventionalcommits.org	The commit message convention in full
Good first issues (for example goodfirstissue.dev)	Find beginner-friendly open-source issues for a first contribution
The Command Reference PDF from this class	Every command explained, plus Actions, workflows, and the panic-button table

Thank you for teaching. Teach the ideas first, the commands second, and keep people typing. Have a great session!